# Breweries Data Lake Pipeline - Technical Documentation

## Table of Contents


1. Introduction ..... 2
2. Project Objectives ..... 2
3. Architecture Overview ..... 2
4. Data Source ..... 3
5. Medallion Data Lake Architecture ..... 3
   5.1 Bronze Layer ..... 3
   5.2 Silver Layer ..... 3
   5.3 Gold Layer ..... 4
6. Data Transformations ..... 4
7. Technology Stack ..... 5
8. Orchestration Strategy ..... 5
9. Data Quality Strategy ..... 6
10. Observability and Monitoring ..... 6
11. Error Handling and Resilience ..... 6
12. Testing Strategy ..... 6
13. Project Structure ..... 7
14. Execution Guide ..... 7
15. Deployment Considerations ..... 8
16. Design Decisions and Trade-offs ..... 8
17. Future Improvements ..... 9
18. Conclusion ..... 9

## 1. Introduction

This document describes the design and implementation of a Data Lake pipeline created to ingest, process, and analyze brewery data from the Open Brewery DB API.

The solution follows modern Data Engineering best practices, including:

- Medallion Data Architecture
- Data Quality validation
- Workflow orchestration
- Scalable data processing using Spark and Delta Lake

## 2. Project Objectives

The main goals of this project are:

- Consume data from Open Brewery DB API
- Persist raw data in a Bronze layer
- Transform and structure the data into a curated Silver layer
- Produce an aggregated analytical dataset in the Gold layer
- Implement orchestration using Apache Airflow
- Ensure pipeline reliability through retries and error handling
- Provide documentation and automated tests for the pipeline

## 3. Architecture Overview

The architecture follows a **layered Data Lake design**, where data moves through refinement stages.

**Data Flow**

Open Brewery API
↓
API Ingestion (Python)
↓
Bronze Layer (Raw JSON)
↓
PySpark Transformations
↓
Silver Layer (Delta Lake — Partitioned)
↓
Aggregation Job

↓
Gold Layer (Delta Lake)

Pipeline orchestration is handled using **Apache Airflow**.

## 4. Data Source

Source: **Open Brewery DB API**

Endpoint used:

https://api.openbrewerydb.org/v1/breweries

The API provides information about breweries around the world, including:

- Brewery name
- Brewery type
- Address
- City
- State
- Country
- Geographic coordinates

## 5. Medallion Data Lake Architecture

The Medallion Architecture organizes data into three layers.

### 5.1 Bronze Layer

Purpose: Store raw data exactly as retrieved from the API.

Characteristics:

- JSON format
- No transformations applied
- Preserves original API payload

Example path: data/bronze/breweries_raw.json

### 5.2 Silver Layer

Purpose: Provide a clean and structured dataset optimized for analytics.

Format: Delta Lake (ACID transactional storage layer).

Transformations applied:

- Schema enforcement
- Column selection
- Removal of null records for critical fields
- Deduplication based on brewery ID
- Partitioning by location

Partitioning strategy:

- country
- state

Example structure:

```
data/silver/
 country=United_States/
  state=California/
```

Benefits:

- Improved query performance
- Efficient data organization
- Scalability

### 5.3 Gold Layer

Purpose: Provide aggregated analytical datasets ready for business insights.

Format: Delta Lake.

Aggregation implemented:

Number of breweries grouped by:

- brewery_type
- country
- state

Output: brewery_count

## 6. Data Transformations

During the Bronze to Silver transformation the following operations occur:

- Read raw JSON dataset
- Apply schema validation
- Select relevant fields

- Remove records with missing location data
- Remove duplicate brewery IDs
- Partition dataset by location

During the Silver to Gold transformation:

- Read Silver dataset
- Aggregate brewery counts
- Group by brewery type and location
- Write results to the Gold layer

## 7. Technology Stack

- Python - main programming language
- PySpark - distributed data processing
- Delta Lake - optimized storage format with ACID guarantees
- Apache Airflow - workflow orchestration and scheduling
- Pytest - automated testing framework
- Requests - API communication

## 8. Orchestration Strategy

The pipeline is orchestrated using **Apache Airflow**.

Pipeline tasks:

1. API ingestion
2. Bronze to Silver transformation
3. Silver to Gold aggregation

Configuration:

• start_date: 2026-03-01
• retries: 3
• retry_delay: 5 minutes

Airflow provides:

- Scheduling
- Retry handling
- Monitoring

## 9. Data Quality Strategy

Data quality focuses on the following dimensions:

- Completeness - Validation that critical fields are not null.
- Uniqueness - Ensuring brewery IDs are unique.
- Validity - Schema validation and field structure verification during data ingestion.

Future Improvements:

- Great Expectations integration
- Data anomaly detection
- Automated data validation pipelines

## 10. Observability and Monitoring

Pipeline monitoring is handled through Airflow logs and execution tracking.

Monitoring includes:

- Task execution tracking
- Failure detection
- Retry monitoring
- Execution time analysis

Alerting mechanisms could include:

- Email notifications
- Slack alerts

## 11. Error Handling and Resilience

Resilience mechanisms implemented:

- API retry mechanism
- Airflow task retries
- Exception handling during ingestion
- Fail-fast validation for critical data quality issues

## 12. Testing Strategy

Tests are implemented using **pytest**.

Current tests validate:

- API availability
- Spark environment functionality

Future improvements may include:

- Transformation tests
- Aggregation validation
- Data quality rule testing

## 13. Project Structure

The project is organized as follows:

```
bees-breweries-case

src/
 ingestion/
 processing/
 utils/

pipelines/

tests/

data/
 bronze/
 silver/
 gold/
```

## 14. Execution Guide

Steps to run the pipeline:

1) Install Python (3.10+)
2) Test python
    a) Open prompt cmd
    b) Type: python --version
    c) Click Enter and verify if returns the installed python version
    d) Type: pip --version
    e) Click Enter and verify if returns the pip version
3) Install Java (recomended OpenJDK 11.0.30)
4) Test Java
    a) Open prompt cmd

   b) Type: java -version
   c) Click Enter and verify if returns the installed Java version
5) Install Apache Spark
   a) Look for something like spark-3.x.x-bin-hadoop3
6) For Windows environments, Hadoop binaries (winutils) are required for local execution.
7) Configure the environment variables
   a) JAVA_HOME – path of the installed folder like this example "C:\Program Files\jdk-11.0.27.6-hotspot"
   b) SPARK_HOME – path of spark folder
   c) HADOOP_HOME – path of Hadoop folder
   d) Edit the variable Path
      i) Including the values:" %JAVA_HOME%\bin", "%SPARK_HOME%\bin" and "%HADOOP_HOME%\bin"
8) It is recommended to install a code editor like Visual Studio Code(VS Code)
9) Clone the GIT repository
10) Open the VS Code
   a) Open the Folder of the cloned repository project in VS Code
   b) Open the terminal in VS Code
   c) Create a virtual environment typing: "python -m venv venv"
   d) Type "venv\Scripts\activate" to start the environment
   e) Type "pip install -r requirements.txt" to install the required libraries
      i) Execute the pipeline typing: "python -m pipelines.breweries_pipeline"

Note: When running on Windows, additional configuration such as HADOOP_HOME and winutils.exe is required due to Spark's dependency on Hadoop native libraries.

## 15. Deployment Considerations

The pipeline was designed to run locally, but could be deployed to cloud environments such as:

- Azure Data Lake + Databricks
- AWS (S3 + EMR)
- Google Cloud

## 16. Design Decisions and Trade-offs

PySpark was chosen for its scalability and industry adoption.

Delta Lake provides:

- ACID transactions
- Schema enforcement
- Efficient analytics queries

The Medallion Architecture ensures:

- Clear data lifecycle
- Improved data quality
- Easier debugging and lineage tracking

## 17. Future Improvements

Potential improvements include:

- Incremental ingestion
- Historical bronze snapshots
- CI/CD pipeline
- Cloud deployment
- Advanced data quality framework

## 18. Conclusion

This project demonstrates the implementation of a modern data pipeline using Python, PySpark, and Apache Airflow, following Data Lake best practices and the Medallion Architecture.

The pipeline enables reliable ingestion, transformation, and aggregation of brewery data from the Open Brewery DB API.